Control: conditionals

⏪ Recap

- Variable: a name tag for a value; `=` assigns
- Function: arguments in, side effect and/or return value out
  - `def` to define your own; `return` to hand a value back
  - Scope: names stay inside their function
- Types: `str`, `int`, `float`; convert with `int()`, `float()`, `str()`
- String methods: `strip()`, `capitalize()`, `title()`

🤔 Scope and `return`: what gets printed?

```
def func1():
    x = 10
    y = 20
    return x + y

def func2(y):
    x = 40
    print(x + y)

a = func1()
print(a)

b = func2(2)
print(b)

x = 20
func2(x)
print(x)
```

Control

- **conditionals**: branching
- loops: repetition

1. Is x less than y?

```
Enter a number: 5
Enter another number: 3
x is greater than y

Enter a number: 2
Enter another number: 2
x is equal to y

Enter a number: 1
Enter another number: 4
x is less than y
```

Comparison operators

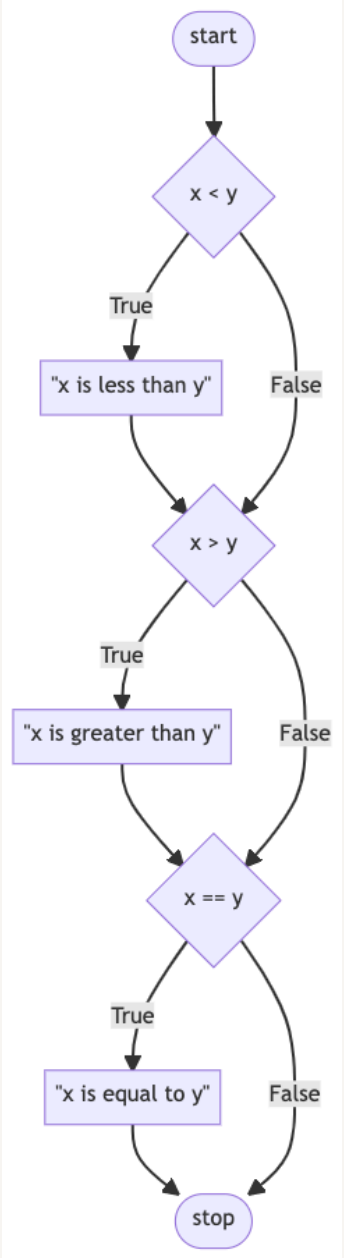
Operator	Meaning
`==`	equal to
`!=`	not equal to
`>`, `<`	greater than, less than
`>=`, `<=`	greater than or equal to, less than or equal to

- `=` **assigns.**
- `==` **compares.**

`if` to run code only when a condition is true

```
x = int(input("Enter a number: "))
y = int(input("Enter another number: "))

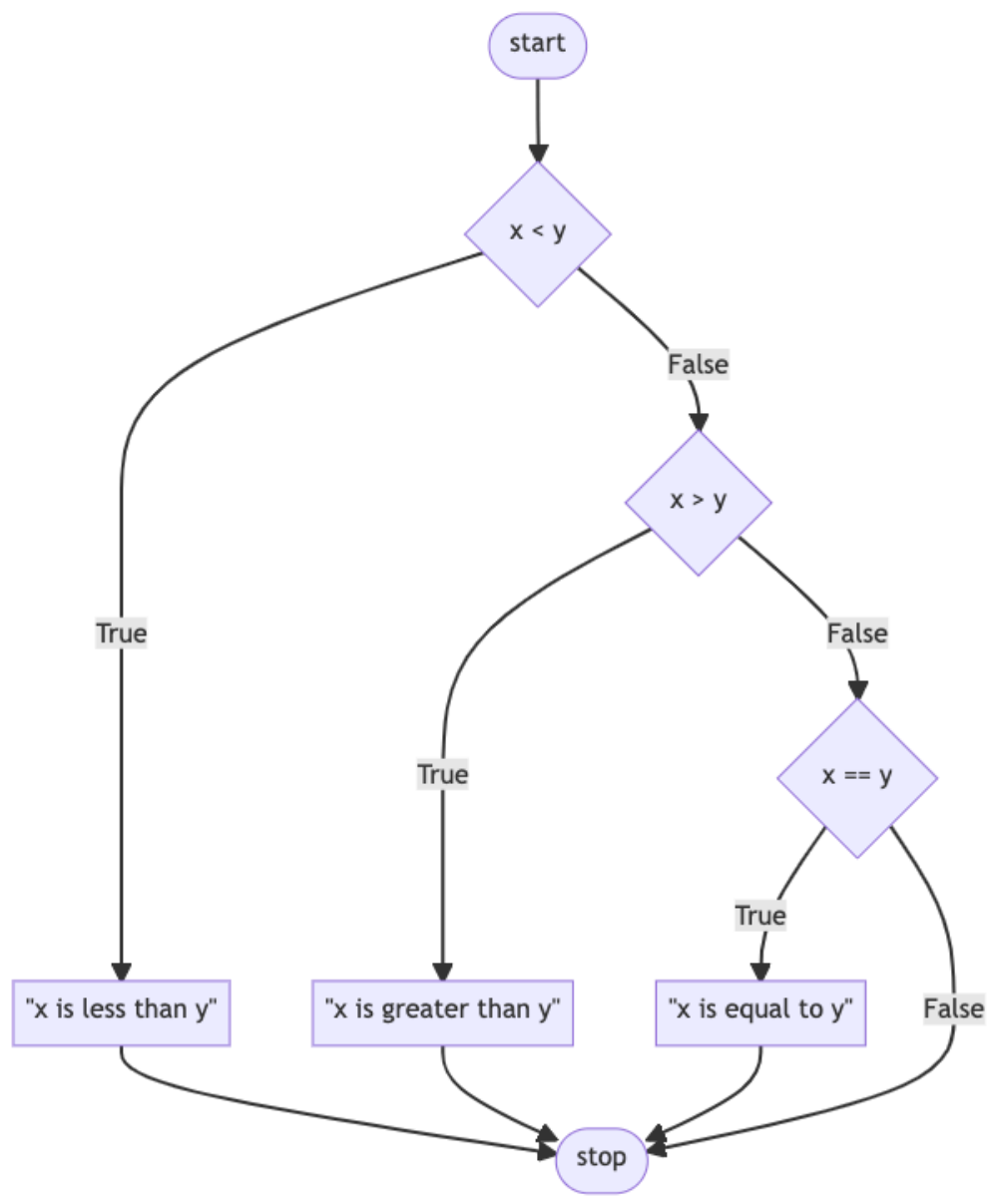
if x < y:
    print("x is less than y")
if x > y:
    print("x is greater than y")
if x == y:
    print("x is equal to y")
```


start
x < y
True
"x is less than y"
False
x > y
True
"x is greater than y"
False
x == y
True
"x is equal to y"
False
stop

`elif` (else if) to check only when earlier conditions were false

```
x = int(input("Enter a number: "))
y = int(input("Enter another number: "))

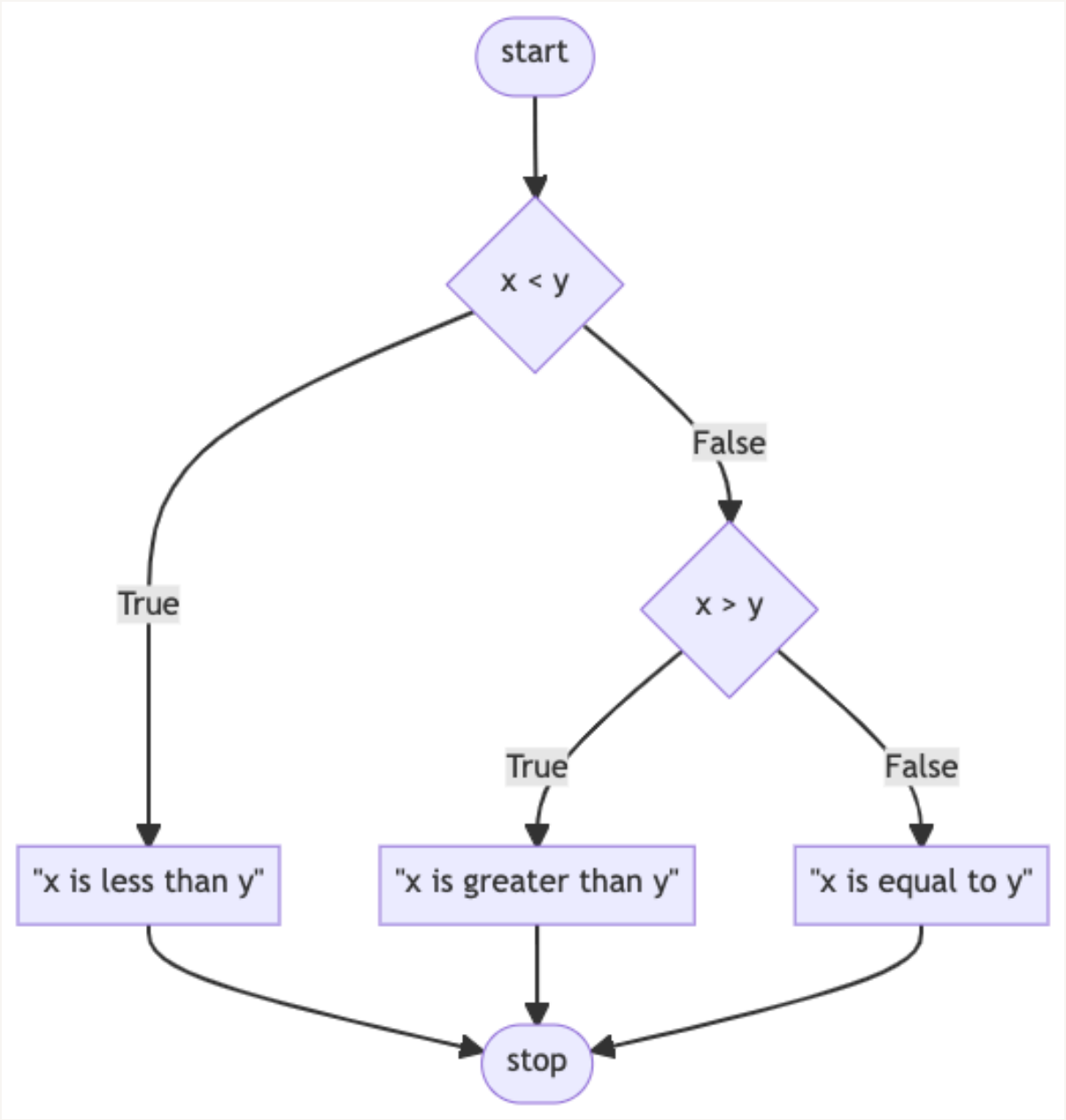
if x < y:
    print("x is less than y")
elif x > y:
    print("x is greater than y")
elif x == y:
    print("x is equal to y")
```


start
x < y
True
False
x > y
True
False
x == y
True
False
"x is less than y"
"x is greater than y"
"x is equal to y"
stop

`else` to catch everything left over

```
x = int(input("Enter a number: "))
y = int(input("Enter another number: "))

if x < y:
    print("x is less than y")
elif x > y:
    print("x is greater than y")
else:
    print("x is equal to y")
```


start
x < y
True
False
x > y
True
False
"x is less than y"
"x is greater than y"
"x is equal to y"
stop

Nested `if` statements

```
x = int(input("Enter a positive number: "))
y = int(input("Enter another number: "))

if x > 0:
    if x < y:
        print("x is less than y")
    elif x > y:
        print("x is greater than y")
    else:
        print("x is equal to y")
else:
    print("x is not positive")
```

💻 2. Find the smallest number

Requirements:

- Prompt the user to enter three different numbers, convert them to integers, and store them in variables named x, y, and z.
- Use **nested if** statements to find the smallest number among the three.
- Print out which variable holds the smallest value with a message like "x is the smallest."

Expected Outputs:

```
Enter a number (x): 5
Enter another number (y): 3
Enter another number (z): 7
y is the smallest
```

3. Is x equal to y?

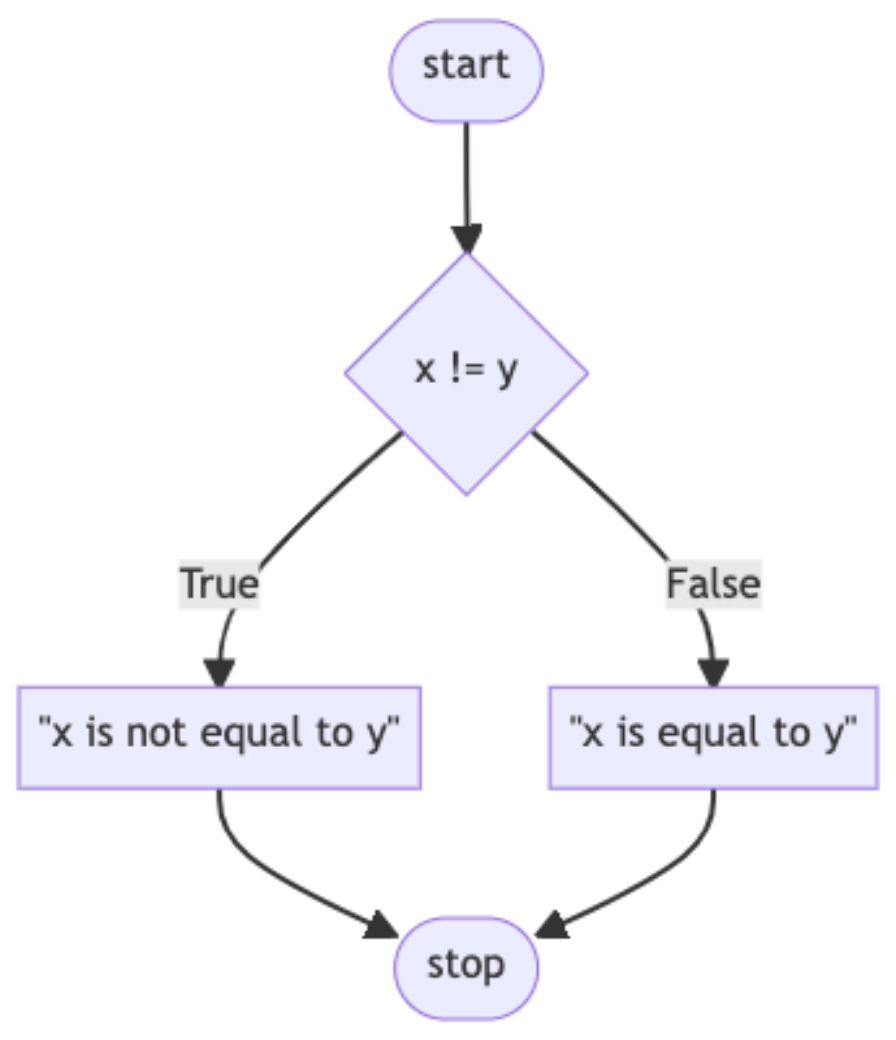
```
Enter a number: 5
Enter another number: 3
x is not equal to y

Enter a number: 7
Enter another number: 7
x is equal to y
```

`!=` (not equal to)

```
x = int(input("Enter a number: "))
y = int(input("Enter another number: "))

if x != y:
    print("x is not equal to y")
else:
    print("x is equal to y")
```


start
x != y
True
False
"x is not equal to y"
"x is equal to y"
stop

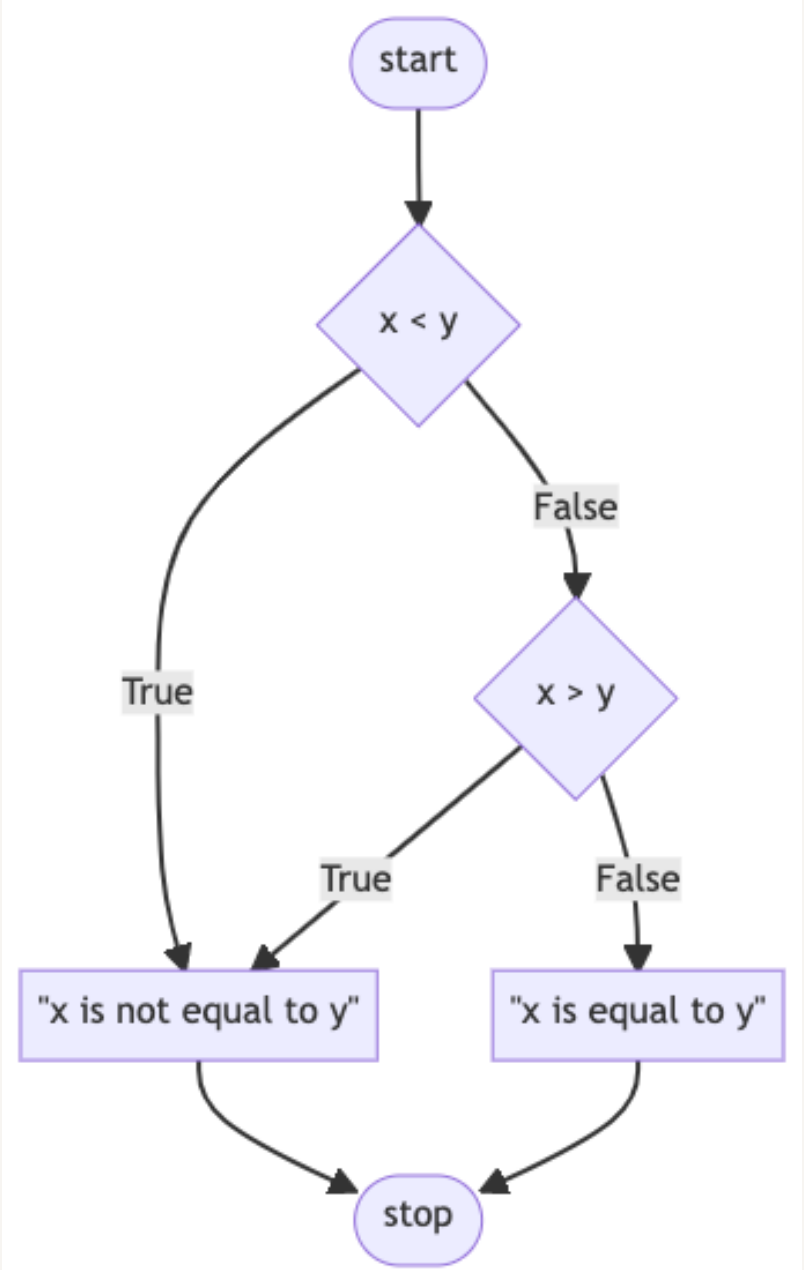
Logical operators to combine conditions

- `or` : either A or B is true
- `and` : both A and B are true
- `not` : negates the truth value

or

```
x = int(input("Enter a number: "))
y = int(input("Enter another number: "))

if x < y or x > y:
    print("x is not equal to y")
else:
    print("x is equal to y")
```


start
x < y
False
True
x > y
True
False
"x is not equal to y"
"x is equal to y"
stop

not

```
x = int(input("Enter a number: "))
y = int(input("Enter another number: "))

if not x == y:
    print("x is not equal to y")
else:
    print("x is equal to y")
```

4. Gradebook

```
Enter your score: 87
A

Enter your score: 72
B

Enter your score: 60
C
```

and

```
score = int(input("Enter your score: "))

if score >= 85 and score <= 100:
    print("A")
elif score >= 80 and score < 85:
    print("A-")
elif score >= 75 and score < 80:
    print("B+")
elif score >= 70 and score < 75:
    print("B")
elif score >= 65 and score < 70:
    print("B-")
else:
    print("C")
```

Compound inequality

```
score = int(input("Enter your score: "))

if 85 <= score <= 100:
    print("A")
elif 80 <= score < 85:
    print("A-")
elif 75 <= score < 80:
    print("B+")
elif 70 <= score < 75:
    print("B")
elif 65 <= score < 70:
    print("B-")
else:
    print("C")
```

One bound is enough: `elif` already ruled out the higher grades

```
score = int(input("Enter your score: "))

if score >= 85:
    print("A")
elif score >= 80:
    print("A-")
elif score >= 75:
    print("B+")
elif score >= 70:
    print("B")
elif score >= 65:
    print("B-")
else:
    print("C")
```

🤔 Why am I not A?

```
score = int(input("Enter your score: "))

if score >= 85:
    print("A")
if score >= 80:
    print("A-")
if score >= 75:
    print("B+")
if score >= 70:
    print("B")
if score >= 65:
    print("B-")
```

💻 5. Find the largest number

Requirements:

- Prompt the user to enter three different numbers, convert them to integers, and store them in variables named x, y, and z.
- Use if and elif statements to find the largest number among the three. [**Hint**: You can use the `and` operator to combine conditions]
- Print out a message identifying the variable that holds the largest value, e.g., "x is the largest."

Expected Outputs:

```
Enter a number (x): 5
Enter another number (y): 3
Enter another number (z): 7
z is the largest
```

6. Parity

```
Enter a number: 4
x is even

Enter a number: 7
x is odd
```

Write `main()` first: what must `is_even()` return?

```
def main():
    x = int(input("Enter a number: "))

    x_is_even = is_even(x)

    if x_is_even:
        print("x is even")
    else:
        print("x is odd")

def is_even(x):
    ...

main()
```

`%` (modulo) to get the remainder

```
x = int(input("Enter a number: "))

if x % 2 == 0:
    print("x is even")
else:
    print("x is odd")
```

Conditions evaluate to `True` or `False`

```
print(2 > 1)           # True
print(7 % 2 == 0)      # False
print(type(True))      # <class 'bool'>

if True:
    print("always get printed")

if False:
    print("never get printed")
```

🤔 A or B?

```
if 2 > 1:
    print("A")

else:
    print("B")
```

🤔 A or B?

```
if True:     # 2 > 1
    print("A")

else:
    print("B")
```

🤔 A or B?

```
if 2 == 1:
    print("A")

else:
    print("B")
```

🤔 A or B?

```
if False:   # 2 == 1
    print("A")

else:
    print("B")
```

🤔 A or B?

```
if 2 > 1 or 2 == 1:
    print("A")

else:
    print("B")
```

🤔 A or B?

```
if True or False:   # 2 > 1 or 2 == 1
    print("A")

else:
    print("B")
```

`is_even()` returns a `bool`: `True` or `False`

```
def is_even(x):
    if x % 2 == 0:
        return True # bool
    else:
        return False # bool

x = 10
x_is_even = is_even(x)

print(x_is_even)
print(type(x_is_even))
```

Return the condition directly: it's already `True` or `False`

```
def is_even(x):
    return x % 2 == 0
```

No variable needed: call `is_even()` inside the `if`

```
def main():
    x = int(input("Enter a number: "))

    if is_even(x): # x_is_even
        print("x is even")
    else:
        print("x is odd")

def is_even(x):
    return x % 2 == 0 # True or False

main()
```

💻 7. Is it odd?

Requirements:

- Write the function `is_odd(x)` to determine if the number is odd or even.
- The function should return True if the number is odd and False otherwise.
- Use an if statement to check the output from `is_odd(x)` .
    - If True, double the number.
    - If False, square the number.

Expected Outputs:

```
Enter a number: 3
x is odd. 3+3 = 6

Enter a number: 4
x is even. 4*4 = 16
```

8. Course code lookup

```
Enter a course code: INSY
Management

Enter a course code: MATH
Science

Enter a course code: XYZ
I don't know
```

String comparison

```
def clean(s):
    return s.strip().upper()

code = clean(input("Enter a course code: "))

if code == "INSY":
    print("Management")
elif code == "ACCT":
    print("Management")
elif code == "FINE":
    print("Management")
elif code == "AEMA":
    print("Agricultural & Environmental Sciences")
elif code == "ECON":
    print("Arts")
elif code == "MATH":
    print("Science")
else:
    print("I don't know")
```

`in` to check membership

```
def clean(s):
    return s.strip().upper()

code = clean(input("Enter a course code: "))

if code in ["INSY", "ACCT", "FINE"]:
    print("Management")
elif code == "AEMA":
    print("Agricultural & Environmental Sciences")
elif code == "ECON":
    print("Arts")
elif code == "MATH":
    print("Science")
else:
    print("I don't know")
```

Return the faculty instead of printing it

```
def clean(s):
    return s.strip().upper()

def get_faculty(code):
    if code in ["INSY", "ACCT", "FINE"]:
        return "Management"
    elif code == "AEMA":
        return "Agricultural & Environmental Sciences"
    elif code == "ECON":
        return "Arts"
    elif code == "MATH":
        return "Science"
    else:
        return "I don't know"

code = clean(input("Enter a course code: "))
print(get_faculty(code))      # the caller decides to print
```

🤔 Does `get_faculty()` check every `if`?

```
def get_faculty(code):
    print("checking Management")
    if code in ["INSY", "ACCT", "FINE"]:
        return "Management"
    print("checking AEMA")
    if code == "AEMA":
        return "Agricultural & Environmental Sciences"
    print("checking ECON")
    if code == "ECON":
        return "Arts"
    print("checking MATH")
    if code == "MATH":
        return "Science"

print(get_faculty("INSY"))
print(get_faculty("MATH"))
print(get_faculty("XYZ"))
```

Wrap up

- `if` / `elif` / `else`: run at most one branch; later conditions are checked only if earlier ones were false
- Comparison operators: `==`, `!=`, `>`, `<`, `>=`, `<=` (`=` assigns, `==` compares)
- Logical operators: `and`, `or`, `not`
- `bool`: conditions evaluate to `True` or `False`
- `%` (modulo): the remainder; `x % 2 == 0` checks for even
- `in`: membership, e.g., `code in ["INSY", "ACCT"]`
- `return` exits a function; a function with no `return` hands back `None`